

MIN – Reinforcement Learning #2

Quentin Surdez
ISCL, HEIG-VD
quentin.surdez@heig-vd.ch

Contents

Introduction	1
Experiences	1
Forzen Lake Problem	1
Question 1.1	1
Question 1.2	2
Question 1.3	3
Question 1.4	6
Question 1.5	8
Taxi	11
Question 2.1	12
Question 2.2	12
Question 2.3	17
Question 2.4	17

Introduction

In this lab, we focus on understanding the concepts behind reinforcement learning. This is done through the use of a library, called gym, that handles the environment in which our agent will evolve. We will use two environments that are pre-created by gym, *Frozen Lake* and *Taxi*.

Experiences

Forzen Lake Problem

In the following section, we will study the Frozen Lake problem, which is a part of the toy series from the library gym. An agent needs to find its way from the source to the goal without stepping in a hole.

Question 1.1

Identify and copy the python code that performs the value function adaptation (i.e., the modification of Q-values). Is it the same as the one presented in the lesson? Does it correspond to SARSA or Q-learning? What is the difference between those learning algorithms? Explain.

Code that performs the value function adaptation:

```
1 next_max = np.max(q_table[next_state,:])
2 new_value = ((1 - alpha) * old_value) + (alpha * (reward + gamma * next_max))
3 q_table[state, action] = new_value
```

I observe a small difference between the formula given within the course and the one above. In deed, the course showcases a version without the adaptation of the old value by the $1-\alpha$ parameter. However in the end it's the Q-learning algorithm. The $(1 - \alpha)$ is just rearranging the equation for quicker computing.

- Q-learning is called off-policy, it will learn the policy without any dependance on the agent's actions. It will update the Q-values of the model based on the maximum possible reward for the next step. The action taken by the agent is not relevant in this context.

Formula for Q-learning:

$$Q(s, a) \leftarrow Q(s_t, a_t) + \alpha(r_{t+1} + \gamma \max_{a'}(Q(s_{t+1}, a') - Q(s_t, a_t)))$$

- SARSA is called on-policy, it will learn the policy with a total dependance on the agent's action. It will update the Q-values of the model based on the actual action taken.

Formula for SARSA:

$$Q(s, a) \leftarrow Q(s_t, a_t) + \alpha(r_{t+1} + \gamma(Q(s_{t+1}, a') - Q(s_t, a_t)))$$

Question 1.2

Using a 4x4 environment modify the code to stop the learning process every 100 epochs to evaluate the performance of the agent. To do this, you can run 100 episodes (Each time the agent is placed in a starting position and tries to reach the target. Take note of how many times it reaches the target) while putting alpha to zero (no learning). Generate a plot of the evolution of the performance as a function of interactions.

I have computed the success rate of our agent every 100 epochs as to evaluate its behavior. The results are presented in Figure 1

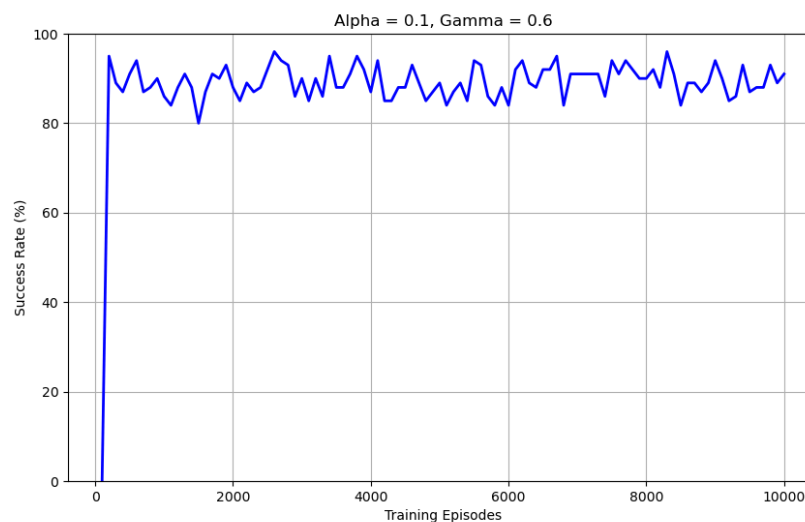


Figure 1: Graph of successes rate as a function of epochs

We can see that the agent quickly reaches high percentage of successes. In deed, the 4x4 environment is deterministic and is quite simple to resolve. However, we can also observe that after the 100th epochs, the success rate of the agent will vary between 98% to 80%. This behavior comes from how the agent is set up. In deed, 10% percent of the time, which is the value of our epsilon, the agent will not choose

the best value in its Q-table for the next step, but it will choose a value randomly. This behavior is set to promote exploration and discover new ways to attain the goal.

In this very simple setting it is not very efficient, however, if we have a less simple environment, it will prove useful as the agent won't be able to rely only on its past experiences, and will need to discover new ways to reach the goal or receive the reward if its current best path leads into a wall for example.

Question 1.3

Modify gamma (the discount factor) to 0.9, and alpha (learning rate) to 0.05 and 0.01. Compare the resulting plots of performance vs. epochs of the 6 combinations of hyper-parameters and provide your observations.

According to my knowledge, gamma is the value of the future reward and changing it will change how the agent is impacted by the future reward. The lower the gamma, or discount factor, is, the more immediate reward are privileged against the future ones. On the other hand, the higher the gamma is, the more the future reward will be considered as just as important as the immediate one.

The learning rate tells the magnitude of step that is taken towards the solution.

Here are the six plots of success rate percentage with the corresponding value of hyperparameter as title:

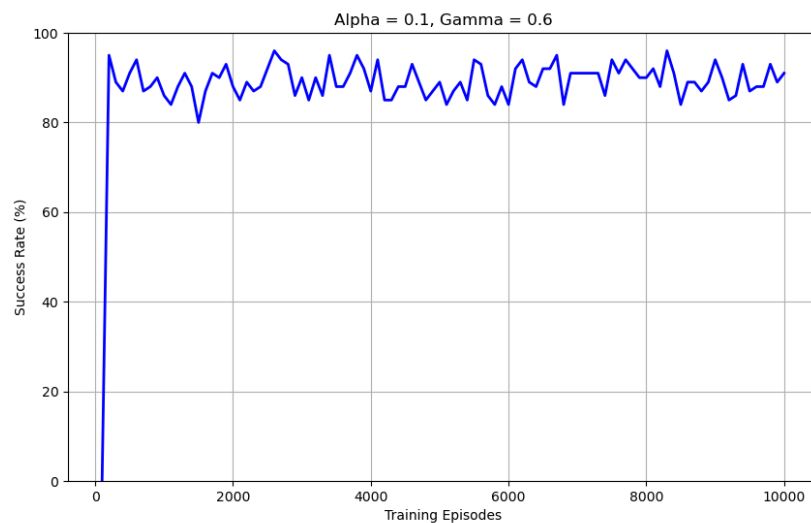


Figure 2: Graph of successes rate with alpha = 0.1 and gamma = 0.6

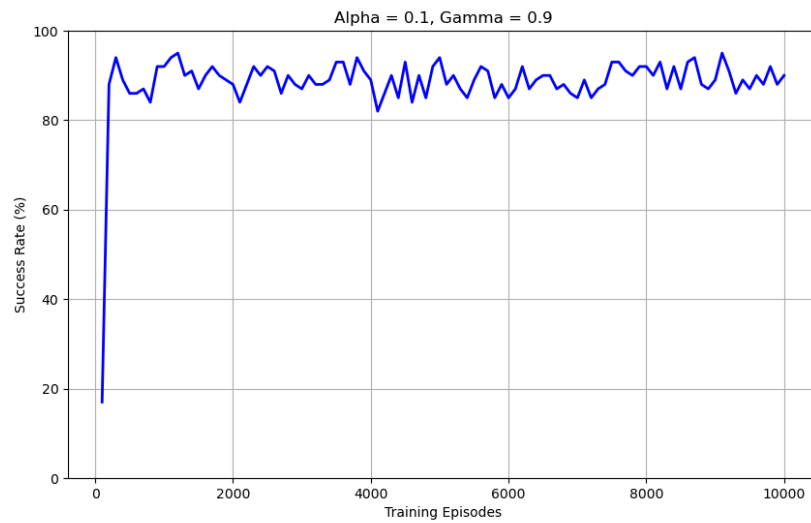


Figure 3: Graph of successes rate with alpha = 0.1 and gamma = 0.9

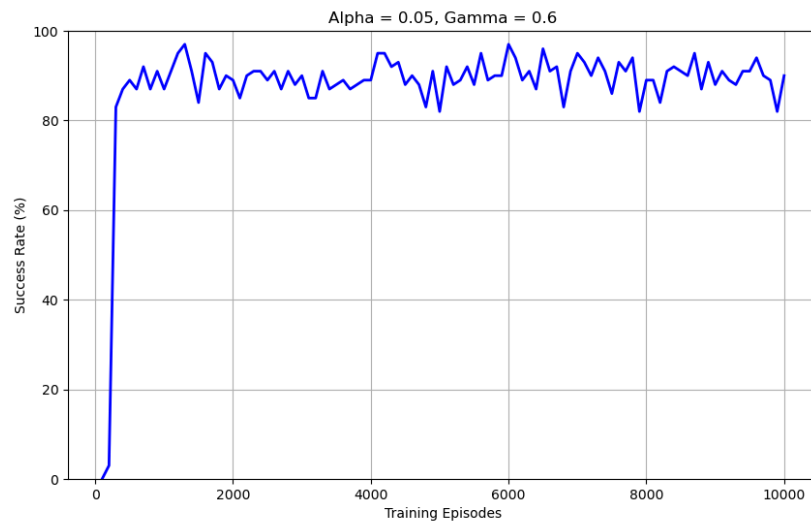


Figure 4: Graph of successes rate with alpha = 0.05 and gamma = 0.6

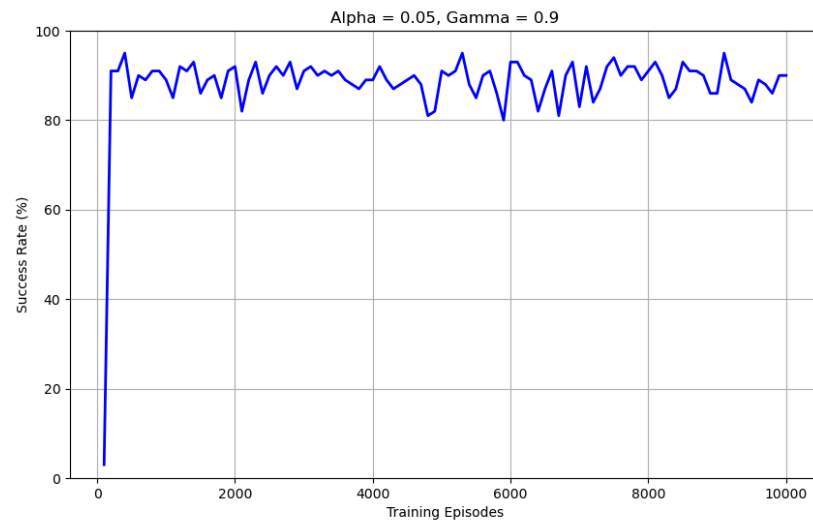


Figure 5: Graph of successes rate with alpha = 0.05 and gamma = 0.9

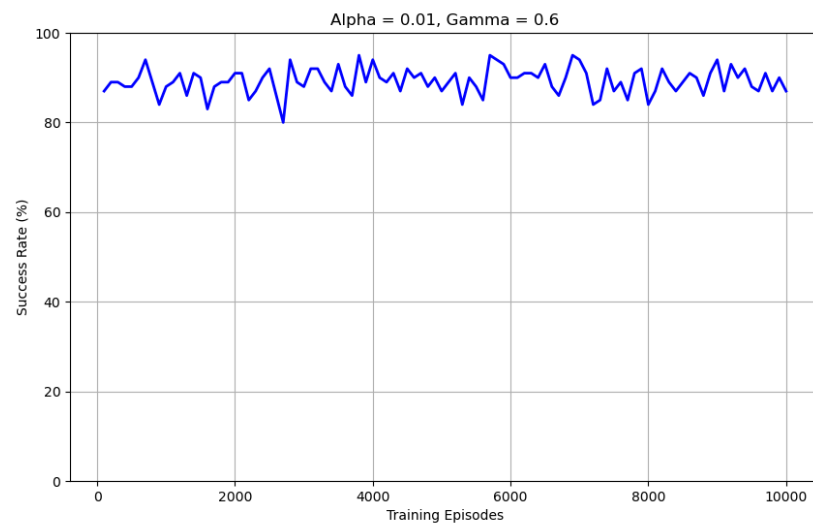


Figure 6: Graph of successes rate with alpha = 0.01 and gamma = 0.6

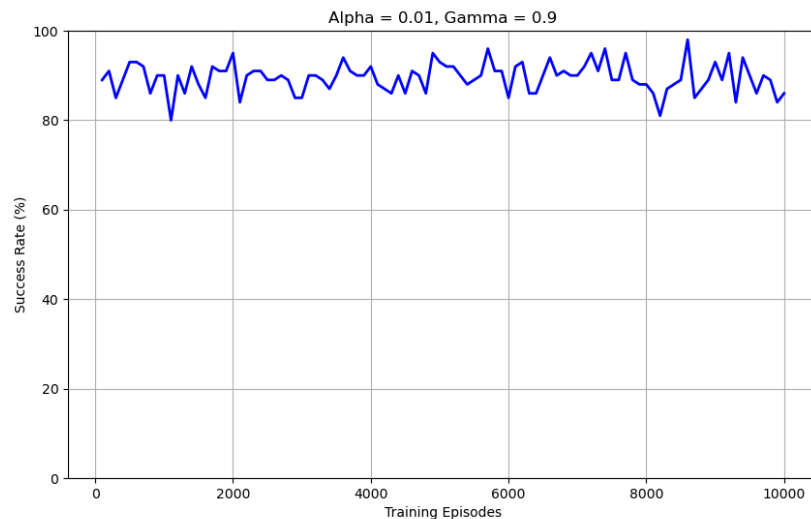


Figure 7: Graph of successes rate with $\alpha = 0.01$ and $\gamma = 0.9$

We cannot see a real difference between the plots where the gamma value changes. This likely due to the fact that the space in which the agent evolves is quite small in this experiment. In deed, it evolves in a 4x4 grid. It is a good hypothesis, in my opinion, that the change of the gamma hyperparameter, would impact the graphs way more if the space in which the agent evolves has more complexity, more possible future states.

The one difference between the graphs that is quite obvious, is how the graphs with the alpha parameter don't start around 0.01 Figure 6 and Figure 7. No points exist between 0 and about 90. This observation is quite interesting as it is the graph with the smallest learning rate. This is likely due to the fact that each experience only minimally changes the Q-values, keeping the random exploration for longer periods and as the space is very small this random exploration finds a path very quickly and thus, even an early trained agent finds the optimal path very quickly. This is a hypothesis based on my current knowledge.

Otherwise, the behavior is quite common and is as explained in the answer of the question 2.

Question 1.4

Run a 5x5 environment. Track the learning process to determine the minimum number of epochs needed to allow the agent to have a good performance. Can it reach a perfect performance ? Is it more rapid that the exhaustive search? Explain.

The method for computing successes rate is the same one as the precedent question. I have chosen to show only a subset of the graphs as there is no observation of real interest to be made. They are very similar, if not the same as the previous ones.

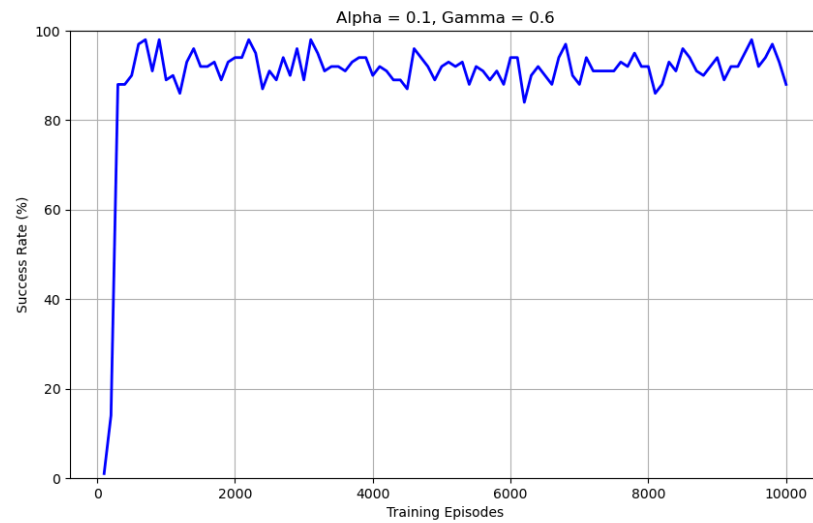


Figure 8: Graph of successes rate with alpha = 0.1, gamma = 0.6, size 5x5

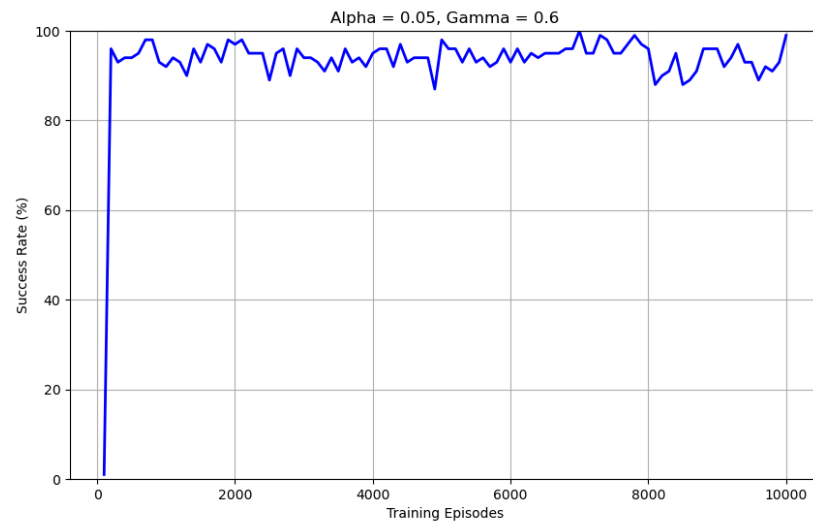


Figure 9: Graph of successes rate with alpha = 0.05, gamma = 0.6, size 5x5

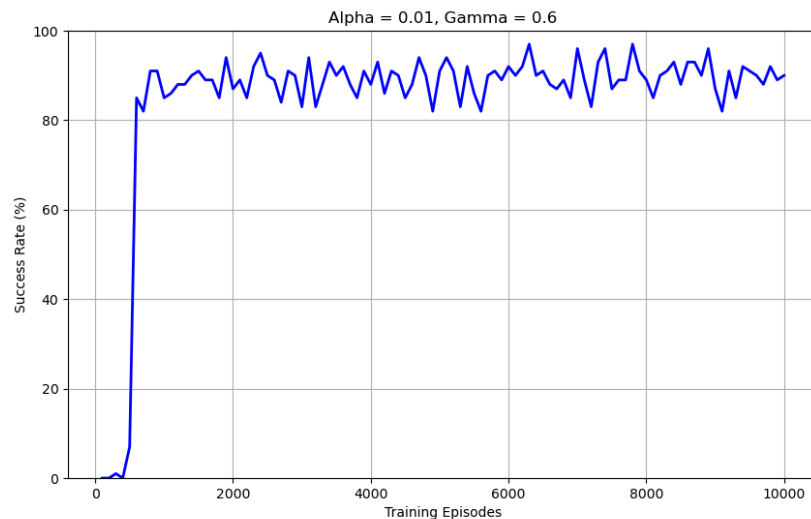


Figure 10: Graph of successes rate with $\alpha = 0.01$, $\gamma = 0.6$, size 5x5

It reaches a good performance very quickly, between 100-500 epochs. As said before, the space lack complexity to see an exploration phase last longer event if we increase the size of the space to 5x5 instead of 4x4. It is way more quicker than the exhaustive search. In deed, it will not search for all the possible paths but only the most efficient one based on its past experiences.

The exhaustive search will only search all possible path until it finds the right one and it is very dependent on the space in which it evolves and won't be able to adapt to change as much as the agent unless it recalculates all possible ways in the new space where the agent can use its past experiences to come to conclusion on the behavior it must take or readapt itself to the new environment.

Question 1.5

Try to modify the reward function by providing the agent with a punishment of $r=-0.1$ for each step it takes to try to reach the target instead of giving it a reward $r=+1$ when it reaches the target. Help: <https://medium.com/@ym1942/create-a-gymnasium-custom-environment-part-1-04ccc280eea9>

It's very interesting how the success rate is impacted. In deed, we can more clearly see the difference between the hyperparameters. The future reward needs to be favored instead of the direct one which will always be -0.1 . So the gamma needs to be higher.

Here are the graphs created:

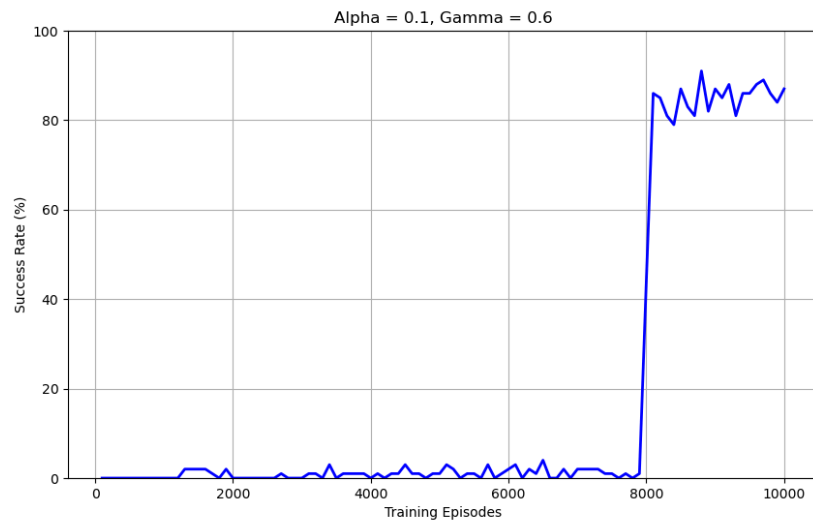


Figure 11: Graph of successes rate with $\alpha = 0.1$, $\gamma = 0.6$ and reward changed

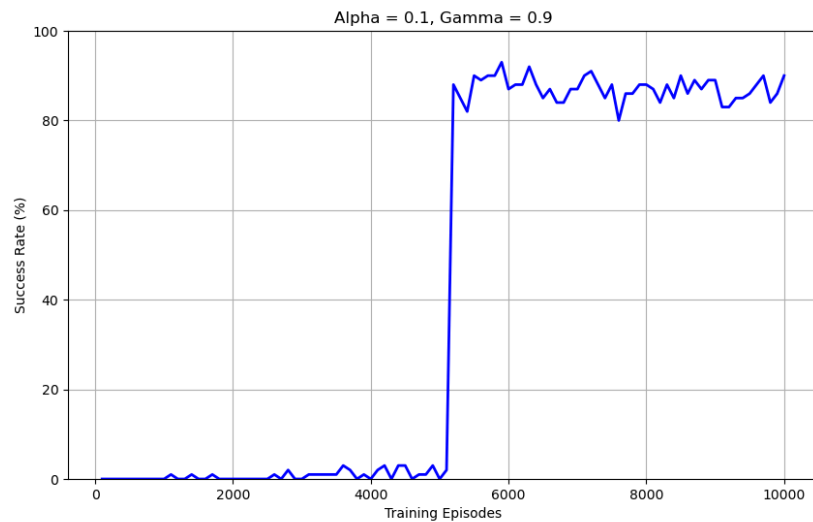


Figure 12: Graph of successes rate with $\alpha = 0.1$, $\gamma = 0.9$ and reward changed

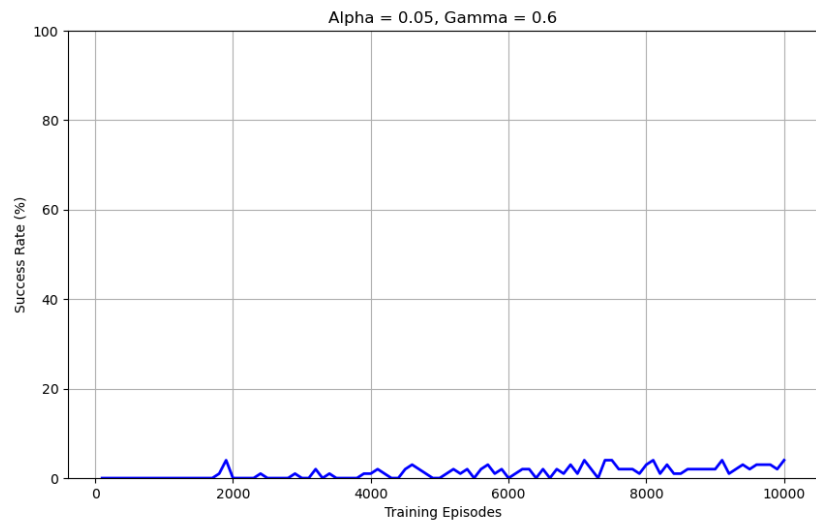


Figure 13: Graph of successes rate with alpha = 0.05, gamma = 0.6 and reward changed

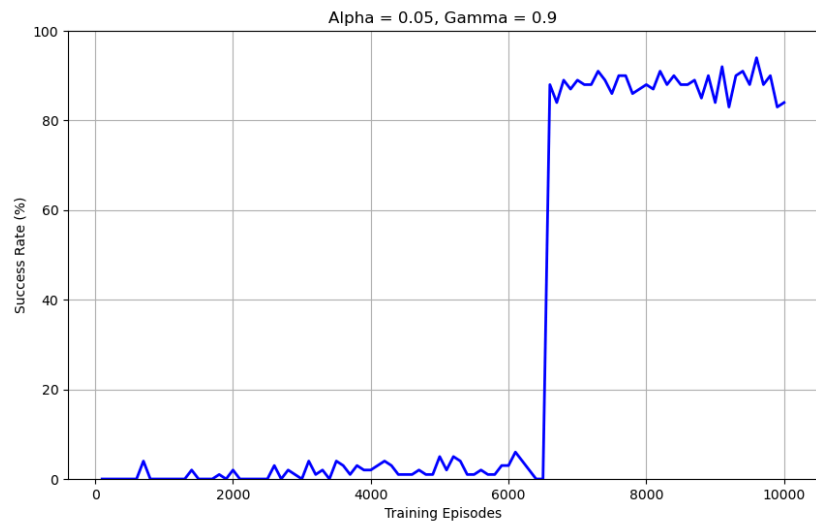


Figure 14: Graph of successes rate with alpha = 0.05, gamma = 0.9 and reward changed

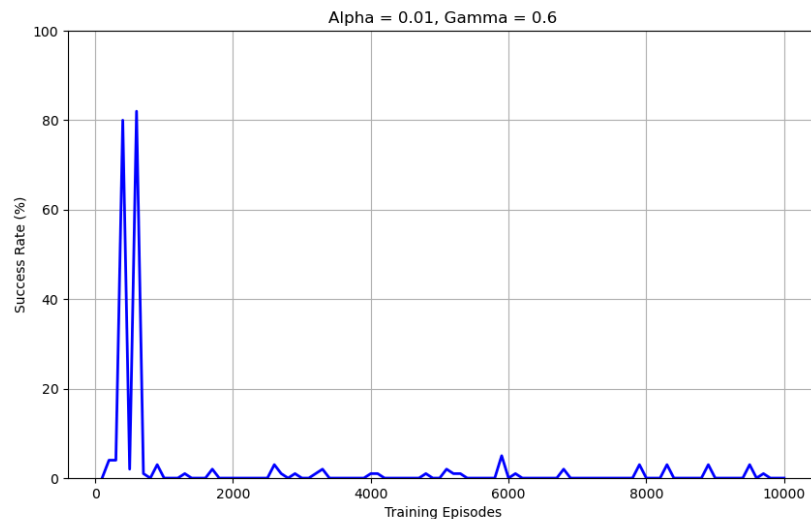


Figure 15: Graph of successes rate with $\alpha = 0.01$, $\gamma = 0.6$ and reward changed

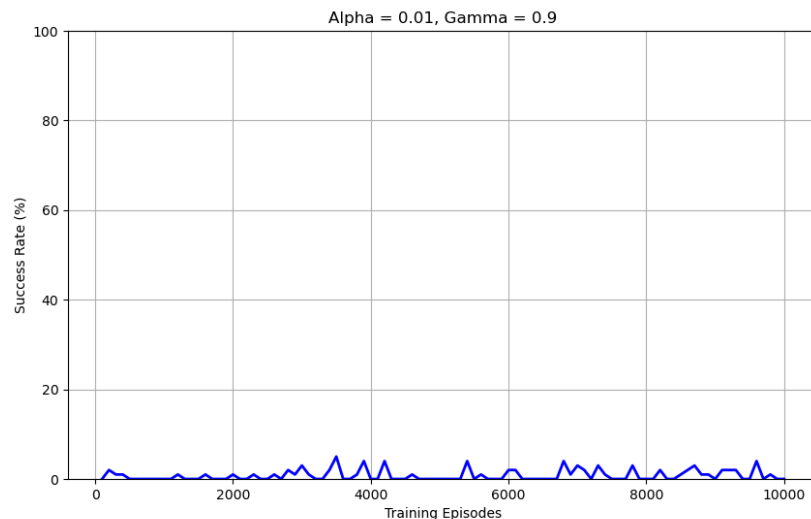


Figure 16: Graph of successes rate with $\alpha = 0.1$, $\gamma = 0.9$ and reward changed

The graph with the most interesting result is the one with the α at 0.05 and the γ at 0.9 (Figure 14). In deed, it finds optimal Q-values at around 6200 epochs. However if the γ is at 0.6 (Figure 13), which will favor current reward and not future ones, we observe that the agent does not find the optimal path. Thus we conclude that γ is clearly important in this situation.

Here the graphs when the α is at 0.01 (Figure 15 and Figure 16), the agent doesn't converge as the α value is too small to have good impact on the learning process.

However, when the learning value is 0.1 (Figure 11 and Figure 12), we can clearly see that the agent finds the optimal solution (100% success rate). However, it finds the solution for the Q-values quicker if the γ is higher as shown on Figure 12, which will value future rewards almost as much as current ones. When the γ value is 0.6 as shown on Figure 11, the future rewards value is decreased, and thus the agent takes more time to find the solution.

Taxi

Here we will study another problem from the toy series of the library gym. We have an agent, the taxi, that needs to go to where the client is, do a specific action, pickup, and then go to where the client wants to be dropoff and do another specific action, dropoff.

Question 2.1

Using the taxi environment modify the code to stop the learning process every 100 epochs to evaluate the performance of the agent. To do this, you can run 100 episodes (Each time the agent is placed in a starting position and tries to reach the target. Take note of how many times it reaches the target) while putting alpha to zero (no learning) and epsilon to zero (no exploration). Generate a plot of the evolution of the performance as a function of interations.

I have plotted the success rate of the agent as a function of the number of episodes run. The resulting figure is

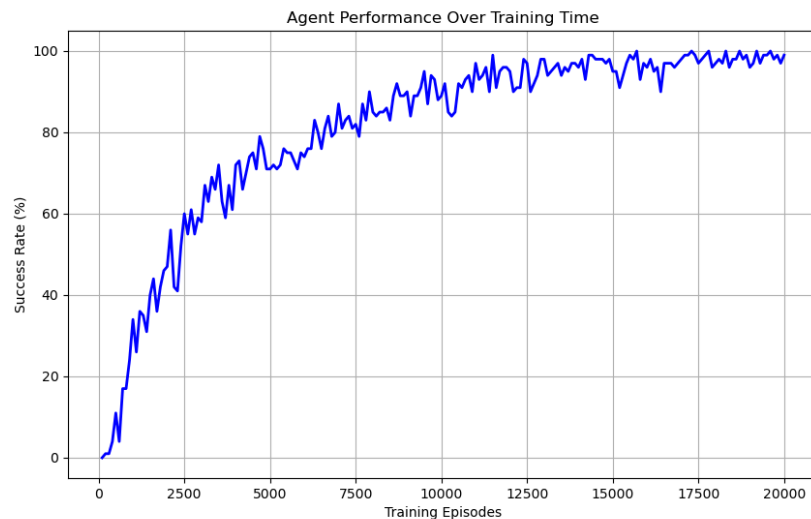


Figure 17: Graph of successes rate as a function of the number of episodes

We can observe that the curve is less steep than the precedent experience. In deed, the range of actions possible as well as the space is far more complex than the Frozen Lake. Thus, the agent takes more time to find the optimal actions to receive its reward of 20. The epsilon is still 0.1, even at the end of the simulation, which explains the fact that even after the agent has found a working solution, it will still do some exploration phases.

Question 2.2

Perform hyper-parameter tuning considering the value of epsilon(exploration), the discount factor and the number of epochs. Present your results.

The method for computing successes rate is the same one as the precedent question. Here are the graph with the different combination of parameters:

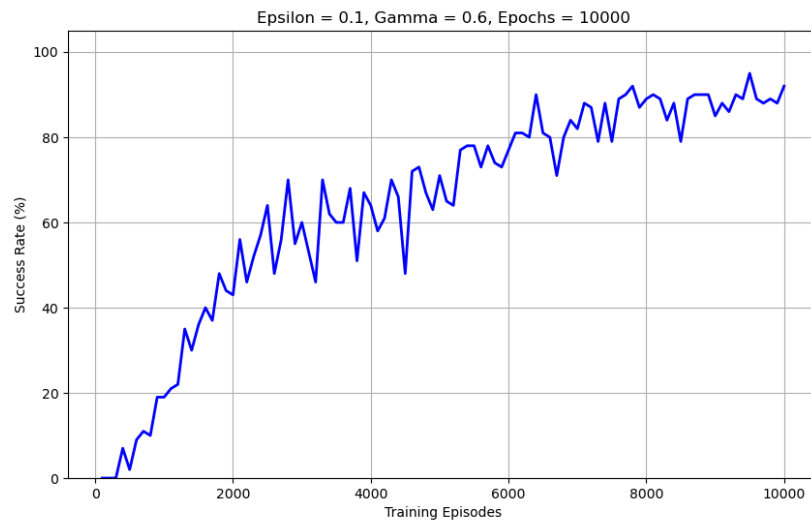


Figure 18: Graph of successes rate with gamma = 0.6, epsilon = 0.1 and epochs = 10000

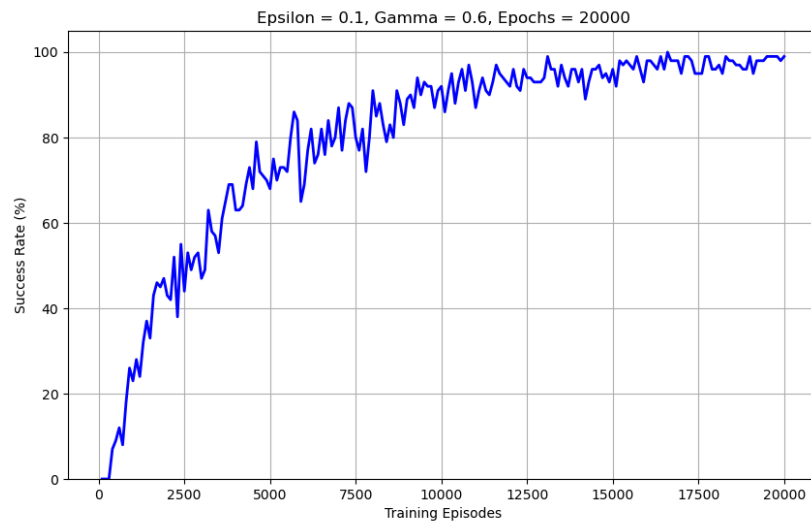


Figure 19: Graph of successes rate with gamma = 0.6, epsilon = 0.1 and epochs = 20000

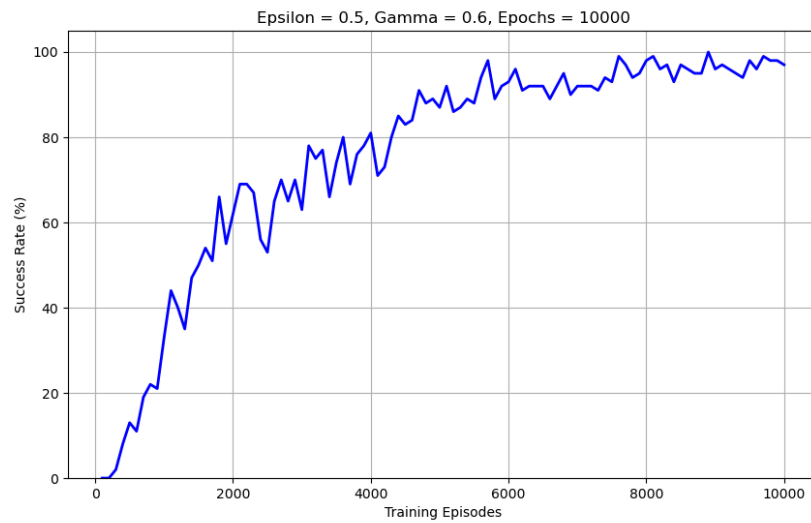


Figure 20: Graph of successes rate with gamma = 0.6, epsilon = 0.5 and epochs = 10000

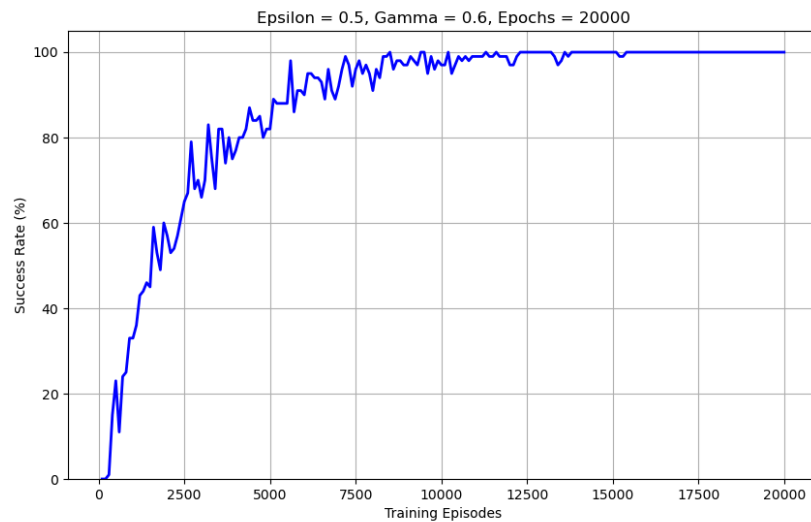


Figure 21: Graph of successes rate with gamma = 0.6, epsilon = 0.5 and epochs = 20000

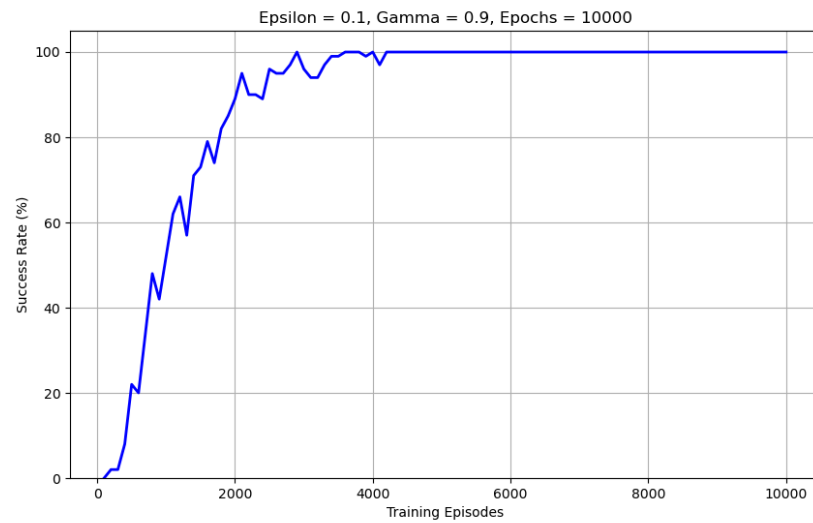


Figure 22: Graph of successes rate with gamma = 0.9, epsilon = 0.1 and epochs = 10000

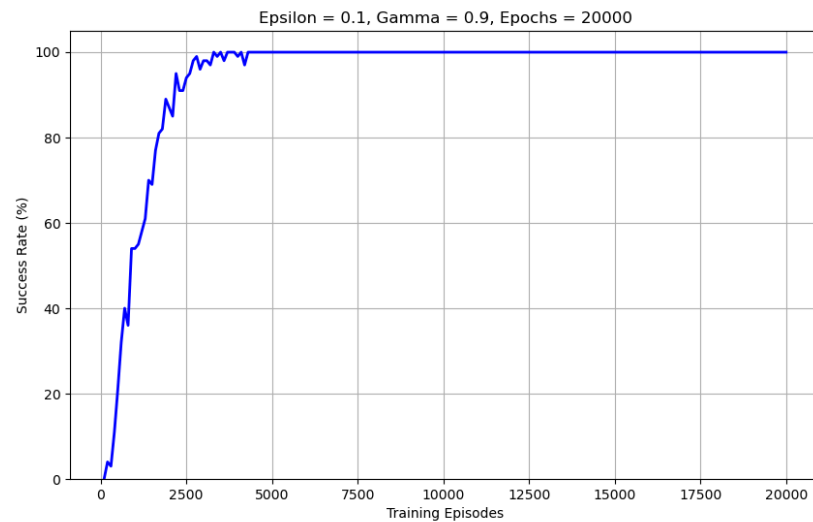


Figure 23: Graph of successes rate with gamma = 0.9, epsilon = 0.1 and epochs = 20000

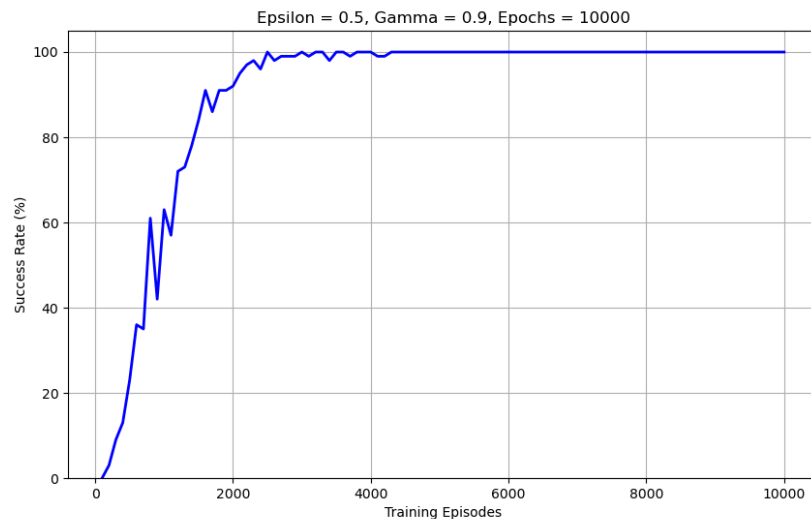


Figure 24: Graph of successes rate with gamma = 0.9, epsilon = 0.5 and epochs = 10000

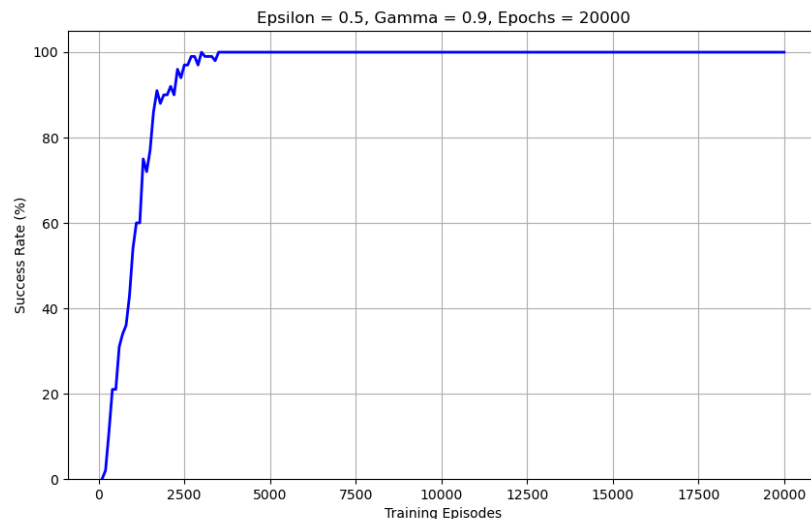


Figure 25: Graph of successes rate with gamma = 0.9, epsilon = 0.5 and epochs = 20000

The results are quite interesting. We can see that when the gamma parameter is equal to 0.9 (Figure 22, Figure 23, Figure 24 and Figure 25), the curve will stay very high up and won't be disturbed by the exploration phase. As told before, this parameter act on how much we value future rewards compared to the current one. With a high value like 0.9, we can conclude that the future reward, the +20 at the end of the simulation, will be almost as highly valued as the current one. This makes me think that the exploration phase, despite being present, won't have much effect as soon as the best Q-values (100% success rate) have been found and recorded in the Q-table.

In these above mentioned graphs, the curve is way steeper than in the other graphs. Again, in this environment, future rewards are important to consider so that the agent can find the best Q-values as quickly as possible. This shows that the convergence is quicker with a gamma value of 0.9 compared to a gamma value of 0.6.

The number of epochs plays an important role as well. We can understand, from the graphs, that when the epochs are fixed to 10000 (Figure 18 and Figure 20), the agent doesn't find a solution for the Q-

values if gamma is equal to 0.6. It takes it around 12500 epochs to find a solution for the Q-table that gives 100% success rate.

Question 2.3

Find how the state of the agent is computed from the observed variables. For a trained agent list the states where the agent performs the “dropoff” and “pickup” actions and verify if the behavior is the right one. You may use the `env.decode()` function. Explain.

The actions are encoded in a simple dictionary where every action has an integer number assigned.

Each state space is represented by the tuple: (taxi_row, taxi_col, passenger_location, destination) An observation is an integer that encodes the corresponding state. The state tuple can then be decoded with the “decode” method.

This how we can check if the agent has learned the rules or not. We check if the constraints of the game are respected and we can see in our case that they are !

The code to check the dropoff and the pick-up states is within the Jupyter Notebook “2q_learning_taxi”.¹

Question 2.4

See the code at the end of the Jupyter Notebook “2q_learning_taxi”.²

¹Claude Sonnet 3.7 has been used to help with coding the different functions created to answer this question.

²Claude Sonnet 3.7 has been used to help with coding the different functions created to answer this question.